How to use GitHub Copilot in DBeaver

The idea is to share the first steps in setting up Github Copilot in DBeaver and enable it to help us give relevant suggestions and usable code.

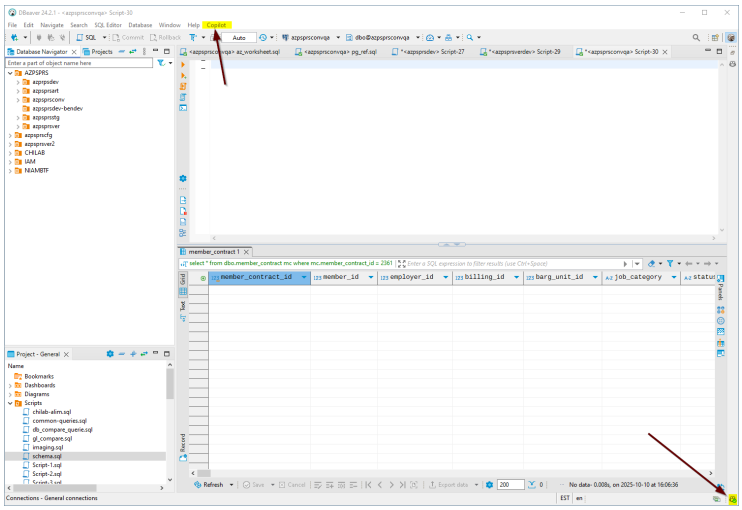
As we use this, I am sure we will uncover more ways to do things more efficiently and better. Please feel free to add/ modify this page as you find anything that will help us use this amazing tool more and better!

Cheers!

- Install
- 2 ways to use:
  - Use Copilot Chat
  - Code Completion
    - 2 Ways to accept suggestions:
      - Fix to accept suggestions easily
- Make suggestions Smarter
  - Create 2 files
    - Generate schema.sql
- Enjoy the much smarter copilot!

Install

- Follow instructions here https://marketplace.eclipse.org/content/github-copilot/help to install Copilot on dbeaver. (Instructions are exactly same for eclipse and dbeaver is Eclipse Rich Client Platform (RCP) application.)
- Once installed you should see the Github copilot button at the bottom right of the dbeaver application



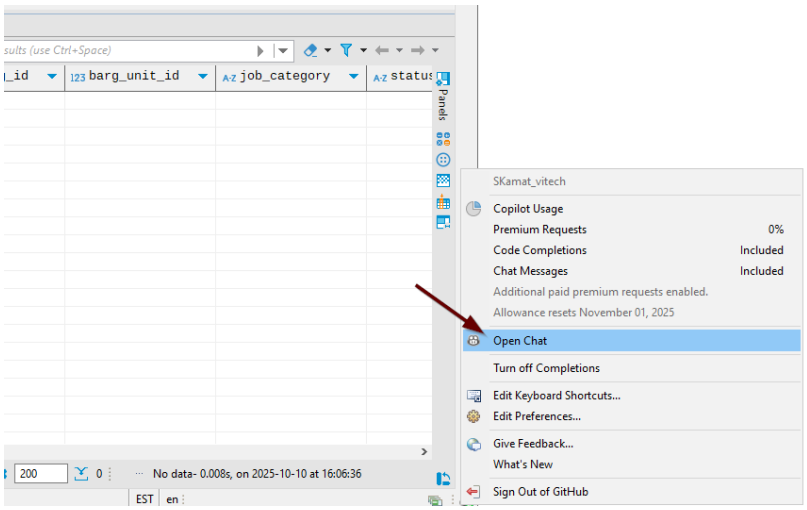
This button is a convenient way to get to the GitHub Copilot Menu.

- Login using your github copilot credentials

2 ways to use:

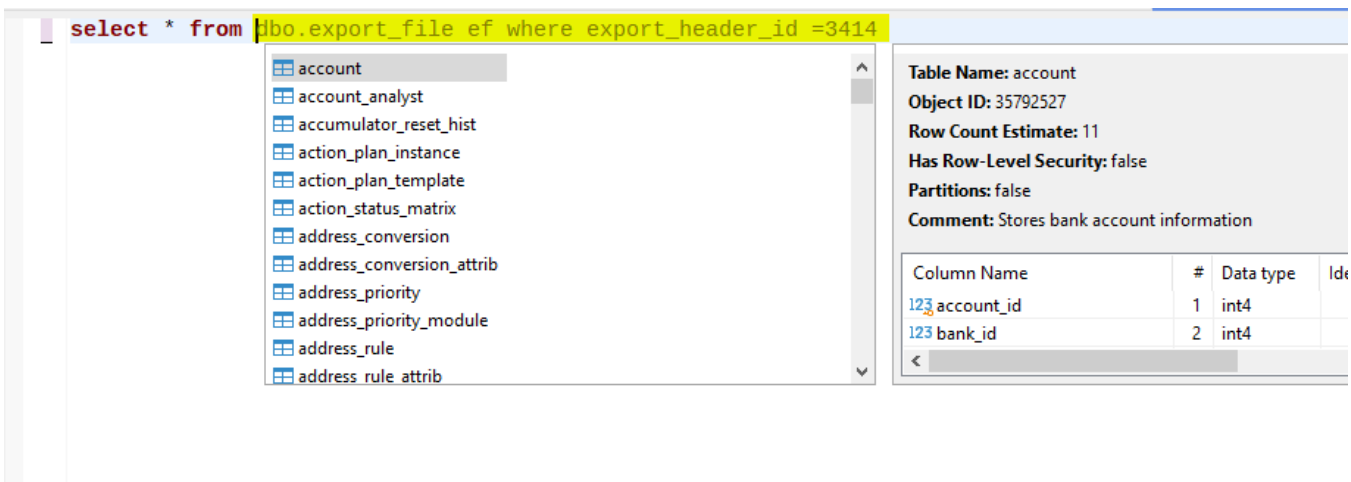
Use Copilot Chat

Chat to ask questions and have copilot write code.



Code Completion

You will also notice that github copilot starts suggesting code completions once you start typing (in gray)



However, note that hitting tab key to accept completions does not work directly in dbeaver.

2 Ways to accept suggestions:

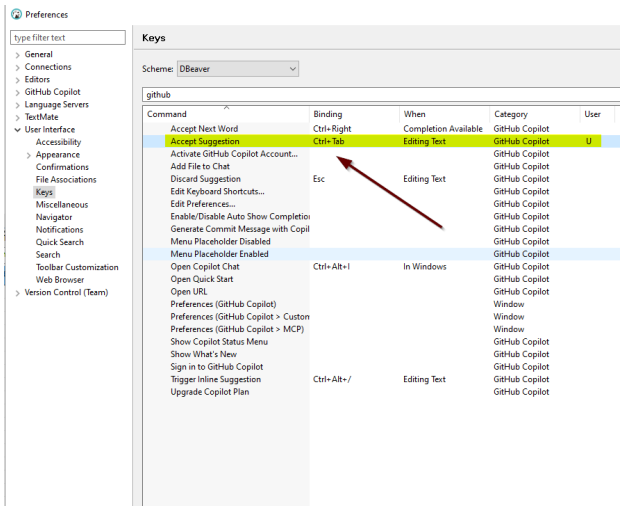
1. You can press Ctrl + Right key to accept suggestions word by word
2. A much better way is to remap your favorite key combination as below, to accept the full suggestion

Fix to accept suggestions easily

Go to Window > Preferences, select User Interface > Keys

Search for Github

Set the key combination for the command 'Accept Suggestion'



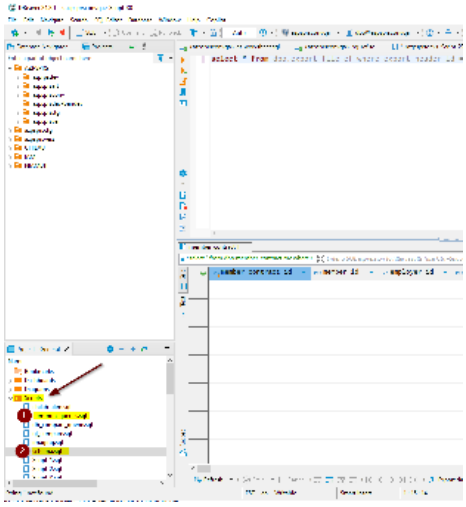
Screenshot shows Ctrl + Tab as this is the combination I picked.

Make suggestions Smarter

You will notice that the suggestions are very generic and have no understanding of the data model or queries you commonly write.

Do the following to make the suggestions smarter:

Locate the project panel on the left hand side and go to the scripts node.

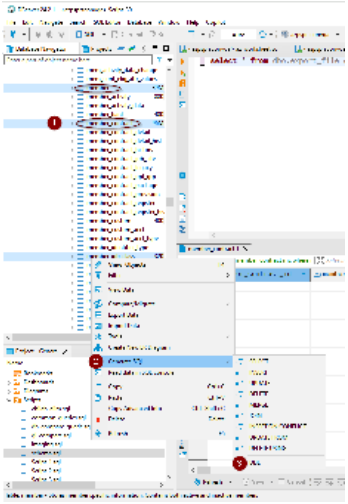


Create 2 files

1. **common-queries.sql** - Paste any and all the queries you have saved for your reference or quick use in this file.
2. **schema.sql** - Copilot expects ddl statements for the tables in this file to understand the data model. Paste ddl for the commonly referenced and any tables you use in your routine work.

Generate schema.sql

An easy way to do this is locate the tables in the dbeaver database navigator pane, select as many using the ctrl button and generate ddl for them at once, copy and paste it in the schema.sql



Enjoy the much smarter copilot!

GitHub Copilot

Your current query in Script-30.sql is:

```
SELECT * FROM person WHERE age > 65;
```

This will work only if the `person` table has a direct `age` column, which is uncommon in normalized databases. Usually, age is calculated from a `date_of_birth` or similar column.

To make your query robust and future-proof, I will:

1. Check if the `person` table has an `age` column or a `date_of_birth` column.
2. If `date_of_birth` exists, rewrite the query to calculate age dynamically using the current date (October 10, 2025).
3. If only `age` exists, keep the query as is.

I'll check the schema.sql file for the definition of the `person` table.

✓ Searching for text in files matching query: person

✓ Searching for text in files matching query: date

✓ Searching for text in files matching query: age

Based on the schema search results:

- There is no evidence of an `age` column in the `person` table in your schema.sql.
- Typically, age is calculated from a `date_of_birth` or similar column, but I did not find a direct match for `date_of_birth` in the search results for "date". However, there may be a column with a different name (e.g., `dob`, `birth_date`, etc.).

To ensure the query is robust, I will:

1. Check the actual definition of the `person` table in schema.sql to identify any date-of-birth-related column.
2. If such a column exists, I will update your query to calculate age dynamically.
3. If not, I will keep your original query.

Next, I will locate the full definition of the `person` table in schema.sql to confirm the available columns.

✓ Searching for text in files matching query: CREATE TABLE person

✓ Reading file schema.sql

The definition of the `dbo.person` table includes a `birth_date` column of type `date`. This means the correct and robust way to select all persons aged above 65 is to calculate their age using the current date and the `birth_date` column.

Here's the improved query for your Script-30.sql file:

```
SELECT *
FROM dbo.person
WHERE birth_date IS NOT NULL
  AND DATE_PART('year', AGE(CURRENT_DATE, birth_date)) > 65;
```

This query:

- Calculates age dynamically using the `birth_date` column.
- Ensures only persons with a non-null `birth_date` are considered.
- Uses standard SQL functions (`AGE` and `DATE_PART`) for accurate age calculation.

I will update your Script-30.sql file with this improved query.

✓ Editing Script-30.sql

I have updated your Script-30.sql file to use a robust query that selects all persons aged above 65 by dynamically calculating age from the birth_date column in dbo.person. This approach ensures accuracy even as time passes and does not rely on a static or potentially outdated age column.

If you need to further filter or join this with other tables (such as contracts or memberships), let me know and I can help extend the query. Your current script is now best practice for this requirement.

I will now read the definition of `dbo.member_contract` from your schema.sql to confirm the relevant columns.

✓ Reading file schema.sql

The `dbo.member_contract` table does not have a direct `person_id` column. Instead, it has a `member_id` column, which likely links to a member, and the member may be associated with a person.

To accurately find all member contracts for `person_id = 1000`, I need to:

1. Identify the table that links `person_id` to `member_id` (likely a `member` table).
2. Confirm the relationship between `person` and `member`.
3. Write a query that joins `member_contract` to `member` (and possibly to `person`) to filter by `person_id`.

I'll search for the `member` table definition and confirm if it has a `person_id` column.

✓ Searching for text in files matching query: CREATE TABLE dbo.member

The `dbo.member_contract` table does not have a direct `person_id` column, but it does have a `member_id` column. To find all member contracts for a specific person, we need to join `member_contract` with the `member` table, which typically links `member_id` to `person_id`.

Since the search for `CREATE TABLE dbo.member` did not return a result, it's possible the table is named differently or defined elsewhere. However, based on standard naming conventions and the presence of `member_id` in `member_contract`, it is highly likely that such a table exists and contains a `person_id` column.

To proceed, I will:

1. Search for any table definition containing `member_id` and `person_id` together, which should reveal the correct linking table.
2. Once confirmed, I will write a query joining `member_contract` to this table to filter contracts for `person_id = 1000`.

Let's search for lines containing both `member_id` and `person_id` in schema.sql.

✓ Searching for text in files matching query: member_id

From the schema search, I found the following relevant information:

- The table `dbo.member` exists and has both `member_id` and `person_id` columns.
- There is a unique index on `(person_id, member_id)` in `dbo.member`, confirming the relationship between a person and their member record.
- The `dbo.member_contract` table has a `member_id` column, which links to `dbo.member`.

Conclusion:
To find all member contracts for `person_id = 1000`, you need to:

1. Join `dbo.member_contract` with `dbo.member` on `member_id`.
2. Filter where `person_id = 1000`.

Here is the correct SQL query for your requirement:

```
SELECT mc.*
FROM dbo.member_contract mc
JOIN dbo.member m ON mc.member_id = m.member_id
WHERE m.person_id = 1000;
```

This will return all member contracts associated with person id 1000.

I will now add this query to your Script-30.sql file, keeping it organized and commented for clarity.

✓ Editing Script-30.sql

Editing Script-30.sql

The query to find all member contracts for person id 1000 has been added to your Script-30.sql file. It